

"set Data Type And Methods"

ex:

let myData = [1, 1, 1, 2, 3];
 عند طباعة myData سيتم طباعة ال array كلها
 let myUniqueData = new set([])
 ال array اللي هضيفها هتترجع
 بالقيمة ال Unique فقط غير متكررة.

عند طباعة myUnique هتترجع [1, 2, 3]
 لمعرفة عدد العناصر الموجودة فيه (.size).

لا حظ: لا يمكن الوصول لعناصر ال set بال index.
 لا حظ: ممكن أضيف كل عنصر على حده فن ال set
 كالاتي: new set().add(1).add(2).
 لا حظ: يمكن حذف العناصر من ال set بـ [.delete()]
 ال value.

لا حظ الفرق بين delete و clear:
 تقوم بحذف جميع العناصر.
 تقوم بحذف عنصر بعينه.

لا حظ: يوجد method اقدر من خلالها
 أتأكد هل العنصر موجود فن ال set وذاك وهي:
 [.has()]
 ال value.

Set Vs WeakSet And Garbage Collector

The weak set is weak, meaning references to objects in a weak set are held weakly.

If no other references to an object stored in the weak set exist, those objects can be garbage collected.

Set

- Can store any Data Values.
- Have size property.
- Have Keys, Values, Entries.
- Can use ForEach.

Weak set

- Collection of Object only.
- Does Not have size Property.
- Does Not have clear, Keys, Values and Entries.
- Can not use ForEach

Usage :

store objects and removes them once they become inaccessible.

أقدر أعمل Loop على العناصر الموجودة في ال Set وأوصل لكل عنصر بالترتيب :

```
Let myset = new set( );
```

```
Let iterator = myset.keys( );
```

```
iterator.next().value → First value
```

```
" ".next().value → next value
```

يقوم بمرور من القيسة عند الوصول لأخر عنصر
ومالة ال set تكون ال done أصبحت
يعني وصلت true والvalue
لتحري عنصر ولا . undefined.

الاستخدام ForEach على ال set :
myset.forEach((e) => console.log(e))

For search :

"weakset Use Cases"

*

*

Map Data Type Vs object

Syntax: new Map (Iterable with Key/Value)

map

object

- Does Not contain Key By Default.

- Has default Key.

- Key Can be anything [Function, object,

- string or symbol.

- Any Primitive Data types].

- ordered by Insertion.

- Not 100% Till

now.

- Get Items by size.

- Need to do manually.

- Can be directly Iterated.

- Not directly And need to use object.Keys()

And so on.

- Better Performance when Add or Remove Data

- Low Performance when comparing To map.

To create new map:

Let myMap = new Map();

لنلاحظ الفرق: `object.create(null);` و `new Map();`

Does Not Contain
Keys By Default.

Does Not Contain
Keys.

لنلاحظ الفرق بين: `object` و `map`:

```
let myObject = {
  10: "number",
  "10": "string"
}
```

```
let myMap = new Map();
myMap.set(10, "number");
myMap.set("10", "string");
```

`myObject[10]` ← عائد number
ولكن عائد string
لأن "10" عملت
override على 10.
الناية.

```
myObject = {
  10: "string";
};
```

`myMap.get(10);` ← عائد number
`myMap.get("10");` ← عائد string

```
myMap = {
  10 => "Number",
  "10" => "string"
};
```

لنلاحظ: `map` يتوافق على ترتيب البيانات على حسب الإدخال التي أننا عملت.

`object` مقدرش أعتمد عليها من ترتيب البيانات.

Map Methods

[set - get - delete - clear - has]

Let myMap = new map();

myMap.set(□, □);

Key ↓ Value

console.log(myMap.get(□));

Key

or

new Map ([

[10, "Number"],

[Key, Value]

]);

set / get

myMap.delete(□);

Key

delete / clear

يقوم بحذف العنصر بال Key الخاص به

myMap.clear();

يقوم بحذف جميع البيانات

الموجودة في ال map

لافل: لو جيبه اظبع

من ال Console هيرجع

False او true

لو العنصر
الحذف
لو العنصر
متحذفش

has

يقوم بعمل check

ال map ليهش فيه

العنصر بال Key الخاص به

موجود في ال map ولا

Map Vs Weak Map

Weak Map Allows Garbage Collector To Do Its Task But Not Map.

Map

Key Can be Anything.

Weak Map

Key Can be object only.

يقوم بحذف البيانات

التي مفقودة عليها Access

ولكن بطريقة automatic

وليس يدويا.

*

Array.from Method

*

#128

Syntax:

Array.from (Iterable, MapFunc, This)

ex: console.log (Array.from ("1234"));

→ [1, 2, 3, 4].

console.log (Array.from ("1234", (n) => n * 2));

→ [2, 4, 6, 8].

Arrow Func

الملاحظة: أن Array.from يرجع Array من
Iteration التي أعطيت له.

Array.CopyWithin Method

الـ Method دي تقوم بنسخ جزء من array من مكان لـ مكان تاني في نفس الـ array.

Syntax:

Array.CopyWithin(Target, start, End);
اختيارية

- Any Negative Value will Count From The End.

- Target

--- Index To Copy Part To.

IF At or Greater Than Array Length Nothing will Be Copied.

start

--- Index To start Copying From.

--- IF omitted = start From Index 0.

- End

--- Index To End Copying From.

--- Not Including End.

--- IF omitted = Reach The End.

ex: Let myArr = [10, 20, 30, "A", "B"]
myArr.CopyWithin(1, 2, 4)
→ [10, 30, "A", "A", "B"]

Array. Some Method

تقوم بتطبيق Function على كل عنصر من عناصر array وتأخذ شرط لو الشرط اتحقق ترجع True

Syntax:

Array.some (callbackFunction (Element, Index, Array), This Argument);

callbackFunc $\Rightarrow$ Function to run on every element on the given array.

- Element $\Rightarrow$ The current Element To Process
- Index $\Rightarrow$ Index of current Element.
- Array $\Rightarrow$ The current Array working with.
- This Argument $\Rightarrow$ Value To use As this when Executing callbackFunc.

Using:

check if element exists in array.
check if number in range.

ex:

```
Let nums = [1, 2, 3, 4, 5, 6, 7];
Let check = nums.some (Function (e) {
    return e > 5;
});
```

console.log(check); $\rightarrow$ True

يمكن أخذ 5 من var كائنات استدعاء يمكن
This Argument

Let myNum = 5;

Let check = (...) {

... e > this;

}; myNum);

*

Array.every Method

*

#131

يقس الكلام في (array.some) بفاف condition
ولكن مش يرجع True، إلا لو انطبق الشرط على جميع
العناصر وليس عكس فقط.

Syntax:

Array.every (callbackFunc (Element, Index, Array)
, This Argument);

ex:

const locations = {

20: "Place 1";

"

"

}

let mainLocation = 15;

let locationsArray = object.Keys (...);

let locationArrayNumbers =

locationArray.map (n) => +n);

```
let check = locationArrayNumbers.  
every (Function (e) {  
  return e > this;  
}, mainLocation);
```

→ False

#13

* Spread Syntax And Use Cases *

هذه عملية تقوم بنشر عناصر الـ data
string في spread ليس الـ string
syntax:

... Iterable → spread operator.
- Allow Iterable To Expand In Place

~~Example~~ Using:

// spread with string ⇒ Expand string
Console.log(..."osama"); → o s a m a
Console.log([... "osama"]); → ["o", "s", ...]

// Concatenate Arrays

Let myArr1 = [1, 2, 3];

Let myArr2 = [4, 5, 6];

Let allArrs = [...myArr1, ...myArr2];

// Copy Array

Let CopyArr = [...myArr1];

// Push Inside Array

```
let AllFriends = ["osama", "Ahmed"];
```

```
let thisYearFriends = ["sameh"];
```

```
allFriends.push(...thisYearFriends);
```

// Use with Math object

```
let myNums = [10, 20, -100, 100, 1000];
```

```
console.log(Math.max(...myNums));
```

// spread with objects $\Rightarrow$ Merge object

```
let obj one = {
```

```
  a: 1;
```

```
  b: 2;
```

```
};
```

```
let obj two = {
```

```
  c: 3,
```

```
  d: 4,
```

```
};
```

```
console.log({...obj one, ...obj two, e: 5});
```

*

*